A Reproducible Benchmark of Relational Database Access-Layer Performance in Express.js: A Configuration-Specific Comparison on PostgreSQL and MySQL

Mateusz Miotk[a,*]

[a]*Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks that are fragmented, PostgreSQL-only, and rarely report the tail (p99). **Objective:** We present a reproducible, byte-equivalence-checked benchmark harness and use it to compare eleven access-layer implementations under one clearly specified operating point on PostgreSQL and MySQL; the harness, not a generalized ranking, is the contribution. **Methods:** An identical Express application is served through nine idiomatic access layers and two tuned native baselines, over five access patterns, on the latest stable library versions current at the July 2026 freeze. We report throughput with client-observed response-time p99 from 25 repeated runs per cell under three operating conditions: equal demand (a fixed 50-connection high-load point), equal utilization (an open-loop sweep), and an equal compute budget. An automated cross-check verifies byte-identical responses before measurement. **Results:** The implementation-and-strategy difference concentrates in relation-materializing patterns: the deep/nested fetch spreads 7.15× (PostgreSQL) and 4.96× (MySQL) between native and the slow-

*Corresponding author.
Email address: mateusz.miotk@ug.edu.pl (Mateusz Miotk)

est ORM, narrowing to 1.68× and 2.01× once every layer runs identical SQL, a compound control that bounds the combined query-strategy-and-formulation contribution without isolating a single mechanism. The read ordering is similar across engines (Spearman $\rho \geq 0.86$, Kendall $\tau \geq 0.71$); aggregation and writes transfer only moderately ($\rho = 0.64$, 0.68). No layer exceeds $\approx$110% of one core; the native driver leads all ten engine-pattern combinations and Prisma 7 sits mid-pack. An earlier version's five-core Prisma result, from a now-superseded generation, did not survive the re-run. Large high-load-p99 gaps are a capacity effect that matched utilization largely closes. **Conclusion:** Access-layer rankings can be version-sensitive, as one retired five-core result shows; the durable contributions are the reproducible, byte-equivalence-checked harness, its three-estimand design, and a checklist of genre-specific benchmarking pitfalls, released as a replication package.

Keywords: database access layer, object-relational mapping, Node.js, reproducible benchmarking, response-time tail (p99), empirical software engineering

---

1. Introduction

Express.js remains the most widely used web framework for Node.js, and almost every non-trivial service built on it eventually reaches a relational database through some *access layer*. The choice is not settled by the language or the framework: access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a naive nested fetch silently issues one query per parent row instead of one for the whole result graph [1], a pervasive anti-pattern in real-world database-backed applications (Section 2). Native drivers expose the wire protocol directly (`pg` for PostgreSQL, `mysql2` for MySQL); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects and manage relationships automatically, following patterns such as Data Mapper and Active Record [2]. Node.js back-end performance under load is itself a rec-

ognized empirical topic [3, 4, 5], but that literature treats the runtime and web framework as the unit of comparison rather than varying the database access layer.

In practice, the most visible performance comparisons are *vendor benchmarks*: plentiful but narrow, published by the maintainers of the compared products, confined almost exclusively to PostgreSQL, and settling on a single metric family rather than reporting throughput and the tail together. In the sources searched, we did not identify one reporting the p99 tail behavior widely held to matter for user-perceived latency under load [6, 7]. The peer-reviewed literature is sparser still, limited to at most three ORMs, confined to PostgreSQL, and, where it benchmarks PostgreSQL against MySQL directly, holding the access layer fixed rather than varying it [8, 9, 10, 11, 12] (Section 2 surveys each).

In the sources we searched (Section 2) this leaves a gap along four axes: (1) no access-layer comparison we found includes MySQL, and no PostgreSQL/MySQL benchmark varies the access layer; (2) no vendor-independent study spans the taxonomy broadly, the broad-coverage comparisons being vendor-authored and academic work covering at most three libraries [10, 11]; (3) we did not identify one reporting throughput and the p99 tail jointly, the sole vendor suite pairing throughput with a percentile stopping at a single P95 at one load point [13]; and (4) none measures client-observed performance through an HTTP framework, academic studies instrumenting query execution inside the application process [11] and the Express-based work comparing runtime optimizations on a single stack rather than access layers [14].

This paper addresses that gap with a single, vendor-independent benchmark (authorship only; Section 3 defines the term): an *identical* Express application served through all nine access layers (two native drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five representative access patterns rather than a measured production workload (point read, keyset range scan, deep/nested fetch, per-author aggregation, and insert; Section 3). Seven layers run against *both* engines under an identical

schema, dataset, and pool size; two tuned native-driver baselines mark what hand-tuned driver use achieves. Every (layer, engine, pattern) combination reports throughput (req/s) and client-observed response-time p99 at the fixed 50-connection high-load operating point jointly, from 25 repeated runs, with a utilization-controlled open-loop tail measured below saturation. The study separates two estimands: the *default-configuration* effect of each layer used idiomatically (RQ1–RQ3), and a controlled *same-SQL* contrast that bounds the residual difference once every layer executes identical SQL — changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness (adapters, seed data, an autocannon-driven [15] load runner, and an automated cross-check that all layers return *byte-identical* responses on the four non-mutating patterns) is released as an open replication package.[1]

Three research questions structure the study, each scoped to the benchmarked implementations under the specified operating point:

RQ1 *(performance difference):* How large is the throughput and response-time-p99 *difference* of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities that a fixed operating point conflates: *capacity* (saturating throughput), *overload latency* (the high-load p99 under equal external demand), and *latency at matched utilization* (each layer driven at equal fractions of its own capacity).

RQ2 *(engine dependence):* Does the observed layer ordering transfer across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(consequential patterns):* For which access patterns is the choice of implementation most consequential in this configuration?

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.6.1 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21428215`.

This paper makes four contributions; the **primary** ones are the harness and the benchmark design, which outlast the pinned versions, while the third is a configuration-specific snapshot of a fast-moving ecosystem:

- An **open, reproducible harness**: a uniform adapter contract, a deterministic seed, a matrix runner with repeated randomized-block runs and paired request streams, a physical write-state reset, a same-SQL control with server-side protocol evidence, a coordinated-omission-corrected open-loop generator with a utilization-controlled latency mode, and an automated **correctness cross-check** asserting **byte-identical** responses across layers on the four non-mutating patterns (Section 3).
- A **vendor-independent, broad-coverage benchmark design** spanning the taxonomy's three tiers (native driver → query builder → six ORMs), **dual-engine** (PostgreSQL and MySQL), measured at the **Express level**, reporting throughput and response-time p99 **jointly**, the combination missing from prior art.
- **Configuration-specific observations** on the current library versions and this host: that no layer's throughput scales with additional application cores and the native driver leads all ten engine-pattern combinations, the idiomatic implementation-and-strategy difference concentrating in relation-materializing patterns; that the read ordering is similar across engines while aggregation and writes transfer only moderately (visible only in a dual-engine design); and direct evidence that such rankings can be **version-sensitive**, one headline of an earlier version of this study retired by one re-run on newer versions (Section 4).
- A practitioner-oriented **checklist of four pitfalls specific to access-layer benchmarking** (Section 5), surfaced during development, several caught by the correctness cross-check and the rest by performance analysis.

2. Related Work

We organize prior work along four axes on which our study differs from it — taxonomy coverage, engine coverage, joint throughput/tail reporting, and vendor independence — summarized in Table 1. We located it with a structured scoping search (not a systematic review) of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (July 2026, over titles, abstracts, and keywords), crossing an access-layer set (*ORM*, "object-relational mapping", "query builder", "database access layer", and the seven library names) with context (*Node.js*, *Express*, *JavaScript*, *TypeScript*), engine (*PostgreSQL*, *MySQL*, "relational database"), and measurement (*benchmark*, *performance*, *throughput*, *latency*) sets, restricted to 2006–July 2026 and followed by backward and forward citation chaining from the closest hits. We retained empirical comparisons of relational access layers or engines and excluded non-empirical, non-relational, and single-library studies. The exact strings, per-source dates, and screening decisions are archived with the replication package (`notes/related-work-search.md`); because this is a scoping rather than exhaustive search, we scope every resulting gap claim to the sources searched.

2.1. Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and the object graphs an application manipulates [16], spanning native drivers, query builders, and ORMs following patterns such as Data Mapper and Active Record [2] (Section 1). Torres et al. [17] survey two decades of ORM patterns; the trade-off they document — developer productivity against a hard-to-predict runtime cost — is the one this paper quantifies. The same question recurs in the polyglot-persistence debate over non-relational stores [18], a choice we hold fixed while varying only the access layer.

2.2. ORM performance and data-access anti-patterns

A parallel line studies ORM performance from the opposite direction, detecting and repairing the inefficient SQL that mapping frameworks emit rather than benchmarking layers. The costliest pattern is the N+1 query problem (Section 1); it and related redundant-query anti-patterns are pervasive across real-world web applications [19, 20], and Shao et al. [21] give an independent taxonomy and detector for database-access anti-patterns. Chen et al. quantify the performance impact of redundant data access in ORM-backed Java systems [22], extending earlier anti-pattern detection work [1]. A complementary strand shows the overhead is often *removable* by query synthesis, lazy round-trip batching, or caching [23, 24, 25]; the JavaScript analogue is mis-sequenced `async`/`await` serializing independent I/O [26]. Closest to our design, Lorenz et al. [27] show the best O/R mapping strategy depends on the database technology, van Zyl et al. [28] that ORM performance is highly sensitive to tuning, and a recent study adds an energy dimension [29]. This literature establishes *where* ORM overhead concentrates (relation-materializing, N+1-prone access) and that it is configurable rather than fixed, which our per-pattern results confirm; it does not compare drivers, query builders, and ORMs head-to-head under controlled load, our aim here.

2.3. Peer-reviewed access-layer comparisons

The peer-reviewed literature that measures access layers head-to-head is narrow. Colley et al. [8] give the earliest systematic cross-language account, benchmarking eight ORM frameworks across Java, C#, Python, and PHP against a common schema, but *deliberately excludes JavaScript*, leaving Node.js, and Express in particular, without an academic baseline for the better part of a decade. Three recent studies partially address this: one, in *Procedia Computer Science*, compares Prisma against optimized raw SQL over eight query patterns on a containerized PostgreSQL setup [9] (cited for its methodology; differing library versions, hardware, workload, and harness mean we neither build on nor dispute its absolute speedup figures); a second compares Prisma, TypeORM, and Sequelize

across four relation types, also on PostgreSQL, by response time, memory, and CPU [10]; and the most recent instruments the same three ORMs on PostgreSQL inside a NestJS service, timing internal query stages rather than client-observed requests [11], independently observing the concurrency-dependent Prisma ranking flip we also report (worst at single queries, best under parallel load). A further Express-based study optimizes a single stack (pooling, batching, caching) rather than comparing access layers [14].

2.4. Database-engine, SQL/NoSQL, and Node.js performance

A second, largely disjoint line benchmarks the layers immediately below and above the access layer without varying it directly. Salunke and Ouda [12] compare PostgreSQL and MySQL head-to-head under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, bypassing any application-level access layer, so their work cannot say how a driver, query builder, or ORM would change it; work contrasting SQL and NoSQL databases [30] likewise measures the store, not the access layer. On the Node.js side, Chaniotis et al. [3] established Node's viability as a web platform, Kuffel and Walter [4] tracked how its back-end performance drifts under sustained load, and do Carmo et al. [5] compared back-end frameworks more broadly. None isolates the database access layer as an independent variable, let alone varies it across the taxonomy studied here.

2.5. Standard benchmarks and benchmarking methodology

Standard benchmarks define how database systems are measured: YCSB [31] for cloud-serving stores and the TPC-C/TPC-E suites for OLTP. These target the *engine* and its transaction mix, saying nothing about the driver, query builder, or ORM through which an application issues work (the variable this study isolates), but they set the methodological bar for controlled, repeatable load. A separate methodological literature explains why the vendor benchmarks' single-metric habit is a problem in its own right: Fruth et al. [7] and

Raasveldt et al. [32] catalog pitfalls specific to database benchmarking (coordinated omission, warm-up, tool-induced distortion) that silently corrupt reported percentiles; Dean and Barroso [6] show why the tail, not the mean, determines user-perceived latency once a request fans out over many components, as an Express request serving a deep/nested fetch does; and Li et al. [33] trace the hardware, OS, and application sources of that tail. Load-testing surveys report such measurement is often ad hoc [34, 35], and repeatability studies find systems results frequently hard to reproduce [36], motivating a shared, reusable harness; the benchmarking literature stresses relevance, repeatability, and fairness [37] and the role of neutral benchmarks in advancing a field [38]. From this literature we take a design choice absent from every access-layer benchmark surveyed above: report throughput *and* tail latency (p99) for the same run, rather than a single metric family chosen after the fact.

2.6. Practitioner and vendor benchmarks

A larger body of evidence comes from practitioners and vendors, each authored by a party with a stake in one of the compared products. Drizzle publishes both a Northwind micro-suite spanning nine targets (`pg`, Knex, Kysely, MikroORM, TypeORM, Prisma, and Drizzle itself, pooled and unpooled) and an official k6-driven load test reporting requests per second, average latency, P95, and CPU [13]. Prisma's own site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any latency percentile [39]. IMDBench, maintained by Gel Data (formerly EdgeDB), adds Sequelize and node-postgres and reports both throughput and latency on three CRUD-style operations [40]. Together these three cover more of the taxonomy than the entire peer-reviewed literature combined, yet share the defects noted in Section 1: vendor authorship, PostgreSQL-only coverage, and a single reported metric family.

2.7. Positioning

In the sources searched through July 2026 (protocol archived with the replication package), no study combined broad taxonomy coverage, both engines,

joint throughput/tail reporting, and vendor independence; each covers at most two or three of these four properties (Table 1). This study combines all four (Section 1), closing the same engine gap separating access-layer studies from engine-only work such as [12].

Table 1: Prior work on relational-database access-layer performance, positioned along the four axes on which this study differs from it. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer.

Study	Layers covered	Engines	Metrics	Independent?
Colley et al. [8]	8 ORMs, 4 languages (no JS)	not stated	query latency	Yes
Procedia CS [9]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	Yes
JCCT [10]	3 ORMs	PostgreSQL	latency, memory, CPU	Yes
JCSI [11]	3 ORMs	PostgreSQL	per-stage latency	Yes
Salunke and Ouda [12]	none (engine only)	PostgreSQL + MySQL	throughput, latency	Yes
Drizzle [13]	9 (broad)	PostgreSQL	throughput, latency, P95	No (vendor)
Prisma [39]	3 ORMs	PostgreSQL	median latency only	No (vendor)
IMDBench [40]	4 ORMs + driver	PostgreSQL	throughput, latency	No (vendor)
This study	**9 idiomatic (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**Yes**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

3. Study Design

Our goal, in Goal–Question–Metric terms, is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying its performance cost *with respect to* throughput and response-time p99 *from the point of view of* an application developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [41]. The design follows established guidelines for controlled experimentation in software engineering [42, 43], crossing seven portable access layers with both engines and all five patterns, plus two engine-specific native drivers and their tuned counterparts `pg-tuned` and `mysql2-tuned` (eleven layers: nine idiomatic, two tuned reference baselines; full rationale in `METHODOLOGY.md`). Two estimands are kept separate throughout: the *default-configuration* estimand (RQ1–RQ3, Section 4), the cost a practitioner incurs from each layer used idiomatically, and a *same-SQL*

(raw-path) estimand, what remains once every layer executes identical SQL through its raw facility — a compound contrast, not a mechanism-isolating decomposition.

A fixed concurrency imposes equal *demand* on layers of unequal capacity and therefore drives them to unequal *utilization*, so RQ1 is characterized under three operating conditions rather than one privileged point. *Equal external demand* is the fixed 50-connection *high-load* operating point — high-load rather than saturated, the throughput knee being demonstrated only for the deep fetch (Supplement Figure S2) while the ten-connection pool, not a per-pattern knee, is the binding constraint (Controls, below) — every layer receiving the identical request stream. *Equal utilization* is the coordinated-omission-corrected open-loop sweep, each layer offered a fixed fraction (50/70/85/95%) of its *own* saturating throughput. An *equal compute budget* is the equal-CPU control, the server confined to a fixed number of cores (Section 4). These answer different questions and need not agree — the high-load condition measures capacity under overload, matched utilization the capacity-normalized per-request latency, the equal budget the compute cost of the same work — so we report all three and say which each result speaks to.

3.1. Factors and treatments

We vary three factors. The *access layer* has eleven levels spanning the taxonomy: the native drivers `pg` and `mysql2`, their tuned counterparts `pg-tuned` (named prepared statements reused across calls) and `mysql2-tuned` (the binary protocol with server-side prepared statements via `execute()`), included as engine-specific tuned reference baselines rather than idiomatic treatments; the query builder Knex; and the ORMs Drizzle (a lightweight, query-builder-style ORM), Prisma, Sequelize, TypeORM, Objection, and MikroORM. We classify a library by its dominant interface, not its self-description (Section 1); Drizzle sits at the query-builder end of the ORM range and Objection layers an ORM over Knex, which the results bear out. The nine idiomatic layers are a representative cross-section of the taxonomy's three tiers, not an exhaustive catalogue; other

libraries (for example Kysely, Slonik, and pg-promise) are omitted. Vendor independence here means authorship only: no evaluated library's maintainers were involved or invited to inspect the adapters, which does not remove the consequential choices any benchmark author makes (library selection, idiomatic-use criteria, schema, pool size, tuning); we disclose these and release the harness for inspection. The *engine* has two levels, PostgreSQL 18.4 and MySQL 9.7.1, and the *access pattern* has five, realized as HTTP endpoints (Table 2). The four native and tuned baselines are engine-specific (`pg`/`pg-tuned` on PostgreSQL, `mysql2`/`mysql2-tuned` on MySQL); the other seven layers run on both, giving $4 + 7 \times 2 = 18$ layer $\times$ engine cells and $18 \times 5 = 90$ measured configurations. Supplement Table S29 records which factors are held constant, which vary, and which are uncontrolled on this single virtualized host — the reason we report results for this configuration rather than as general library performance.

Table 2: The five access patterns and what each stresses.

Endpoint	Pattern	Stresses
`GET /posts/:id`	point read (PK)	per-query overhead
`GET /posts?before=`	keyset range scan	index seek + hydration
`GET /posts/:id/thread`	deep/nested fetch	join strategy, N+1 avoidance
`GET /authors/:id/summary`	aggregation	grouped counts / raw SQL
`POST /posts`	insert	write path

3.2. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` $1{-}*$ `posts` $1{-}*$ `comments` $*{-}1$ `authors`), exercising a one-to-many relationship and a two-hop join for the deep fetch. The five endpoints implement five representative access patterns [31] (Table 2): the keyset page issues `WHERE id < cursor ORDER BY id DESC LIMIT` n against the primary-key index rather than a large `OFFSET`; the deep/nested fetch returns a post with its author and every comment, each with its own comment-author; and the aggregation reports post count, comment

count, and total views per author. The database is loaded from a deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) by the native driver, independently of any layer under test, so every engine receives the same seed *data payload* (byte-identical row values); the on-disk physical representation, index layout, and collation naturally differ between engines. The working set fits in memory, so the measurements emphasize the access layer's per-request instruction, protocol, and mapping cost with disk-read latency largely removed; engine-side query execution, buffer management, locking, and logging remain part of every measurement [44]. Every request draws its target identifier uniformly at random from the full seeded range (posts 1–100,000, authors 1–2,000), so load spreads across the whole working set rather than hitting one hot record, and the read endpoints measure warm access-layer cost. A skewed, production-like key distribution is a planned variant (Section 6).

3.3. The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the Express server and load runner are layer-agnostic, and each returns an *identical result shape*. Each uses that layer's *recommended idiomatic* mechanism to avoid the N+1 query problem on the deep fetch: a hand-written join for the native drivers, Knex, and the query-builder-style Drizzle (the tuned baselines reuse the native join unchanged), and relation eager loading (`include`, `withGraphFetched`, `populate`, or `relations`) for the data-mapper ORMs. The comparison is therefore one of each layer's *documented default configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL each layer generates, its round-trip count, and how it materializes results. One asymmetry is inherent to any driver-versus-ORM comparison: the native idiom is hand-authored SQL, the ORM idiom the library's default. Server-side statement logging captures the exact SQL, round-trip count, and `EXPLAIN` plans for the native statements (released with the package); Supplement Table S2 tabulates the per-endpoint counts, which differ by design — the deep fetch takes one query in Sequelize and MikroORM, two

in the native drivers, Knex, Drizzle, and TypeORM, and four in Prisma and Objection. The idiomatic API was fixed by a rule decided *before* measurement — the eager-loading API each layer's official documentation presents first for the pinned version — so the treatment is documented, not editorial. Documentation order is a reproducible selection heuristic, not evidence that the chosen API is performance-optimal or the most common production choice, so we report a documented default-configuration estimand rather than "the performance of each library used idiomatically"; Supplement Table S16 and `METHODOLOGY.md` record, per layer, that API with its **documentation page and version**, its round-trip count, the documented alternative we did *not* use, and that query logging, hooks, and validation are off. The six ORMs split both ways on the join-versus-select-in default; a loading-strategy sensitivity check (Section 4) switches join-default data-mapper ORMs to select-in wherever the alternative is a byte-identical drop-in. To bound the combined contribution of query strategy and formulation, every adapter also implements a *same-SQL control*: identical two-statement deep-fetch SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping, executed through the layer's raw-SQL facility, mirroring the aggregation control (Section 4). Server-side capture, synchronized to a request marker so connection handshakes are excluded, confirms the two control statements are byte-identical across every layer on both engines; what differs is the wire protocol each driver negotiates — on PostgreSQL every layer uses the extended protocol with server-side prepared statements except MikroORM (simple protocol, client-side literal inlining), and on MySQL every layer uses the text protocol except `mysql2-tuned` and Prisma (binary protocol, server-side prepared statements). To ensure no adapter silently returns less, or issues N+1 queries, an automated cross-check (`bench/verify.mjs`) funnels every adapter's rows through shared canonical constructors and asserts full JSON byte equality against the native-driver baseline, on both engines, across 12 probes per adapter spanning the four non-mutating patterns and the same-SQL control, before any timing is collected. This check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; see Section 5).

3.4. Controls

Four controls isolate the access layer as the only variable. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison measures access-layer overhead rather than pool tuning, a known web-application performance factor [45, 46]; for Prisma, whose default pool is otherwise derived from core count, this is pinned explicitly via the `connection_limit` URL parameter. Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests are always waiting in each library's connection-acquisition queue; this queueing is deliberately part of what we measure and is held identical (same pool size, demand) across layers. A 200 ms connection-pool sampler corroborated this on the deep fetch: the native PostgreSQL cell ran with its ten-connection pool fully occupied (mean utilization ≈ 10 of 10) and a mean of about 32 requests pending in the acquisition queue (peaks of 39–40). *(ii) Identical query semantics*: each endpoint issues the same logical query across layers; aggregation uses a single correlated-subquery formulation everywhere (the documented raw-SQL path), so it measures layer overhead on an identical plan, not differences in generated SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, not merely once per cell; the write endpoint is additionally measured in its own dedicated server boot, taken after the database's physical state is rebuilt rather than merely emptied by row deletion (PostgreSQL: drop and recreate the database from a seeded template; MySQL: delete the benchmark rows, rebuild the table with `OPTIMIZE TABLE`, and re-pin `AUTO_INCREMENT`), so sequences, physical layout, and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.5. Measurement procedure

Load was generated with `autocannon` [15], an HTTP/1.1 load tester issuing requests continuously at fixed concurrency and recording a latency histogram; it is a closed-loop (constant-concurrency) generator [47]. Each of the 90

configurations was measured as 25 *repeated runs*: for each replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up whose measurements were discarded, then took one twelve-second measured run. The warm-up length follows a separate cold-boot measurement for a fast, middling, and the slowest PostgreSQL layer: every layer reached and sustained at least 95% of steady-state throughput within 12 s (Prisma by 5 s, native by 9 s, MikroORM by 12 s), absorbing JIT compilation, pool fill, and plan caching, since managed runtimes do not always reach a stable steady state promptly [48]. The experimental unit is the freshly booted server run for one (layer, engine, endpoint) cell in one replicate; individual HTTP requests are subsamples, not independent units. Booting a fresh process per replicate prevents persistent application-process state (JIT state, pool contents, heap) from carrying across replicates, though not full independence: all runs share one host and one short campaign, so replicate index and measurement order are treated as blocking factors. Cell order was reshuffled independently within each replicate; a forward-versus-reversed-order check on the write endpoint (5 replicates per direction) found no detectable history effect on the read cells (reversed-to-forward ratio 0.977–1.280, median 1.012; the wide tail is confined to the noisy MySQL inserts, where five replicates cannot separate order from write noise). Within a replicate, every layer and engine is driven by the identical request-id sequence from a PRNG seeded on the (endpoint, replicate) pair; this paired-stream design removes between-layer sampling noise as a confound, and the first ids of every stream are archived. We report the median of the 25 replicate values; 449 of the 450 dedicated write-endpoint boots completed normally, the exception being one drizzle/MySQL write replicate whose server boot did not clear the post-reset health check, leaving that cell with 24 of 25 runs (the runner records and skips such failures rather than retrying silently). Each replicate yields throughput (requests per second) and latency percentiles p50, p90, p97.5, and p99. `autocannon` records a per-run latency histogram at millisecond resolution; even the slowest deep-fetch cells ($\approx$ 480–516 req/s over the 12 s window) complete $\approx$ 5,800–6,200 requests

per run, so each run-level p99 is estimated from roughly 60 requests in its upper 1% tail, and the reported p99 is the median of the 25 such run-level values. A 60 s re-measurement of the deep fetch (five times the requests) preserves the p99 ranking exactly on both engines and barely moves each cell's p99 (Supplement Table S23), so the 12 s window is adequate for the tail comparison. During measured runs we sampled server CPU and peak resident memory from `/proc` over the full process tree (parent plus descendants, so an in-process engine or spawned helper cannot escape measurement), with database and load-generator CPU sampled separately; a `perf_hooks` GC observer (event count, total pause time) and the 200 ms connection-pool sampler ran inside the server, polled through a `/stats` endpoint after each run. Inserts were measured under each engine's *default* durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`, `full_page_writes=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`, `log_bin=ON`), the regime a deployment runs unless an operator deliberately relaxes it. Each insert is an autocommit single-statement `INSERT` under each engine's default isolation level (PostgreSQL Read Committed, MySQL Repeatable Read); the transactional variant wraps a post and its five comments in one transaction. A labelled secondary run instead relaxed every one of those settings on both engines (10 replicates, write endpoint only) to isolate the durability mechanism (Section 5, Supplement Table S3). A fixed-payload `/baseline` endpoint returning a seed-realistic thread-shaped payload measured the shared Express-plus-serialization floor (bounding framework and serialization cost, not per-request object construction): 10,909 (PostgreSQL server) and 10,921 (MySQL server) req/s, roughly 3× the fastest deep-fetch cell, so the framework floor leaves between-layer spreads visible rather than compressed. To characterize behavior under load, we swept the deep/nested fetch (the most layer-sensitive pattern) across connection counts from 1 to 256 for every layer and engine, since throughput and contention scale non-linearly with concurrency [49]; a separate equal-CPU-budget control (1, 2, or 4 cores) attributes part of that scaling directly to CPU (Section 4). A native, `undici`-

based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring latency from each request's intended send time and clipping timed-out requests into the distribution at their cutoff; this coordinated-omission-corrected design [50, 51] independently validates the closed-loop tail results (Section 4).

3.6. Analysis

We separate a small set of *primary* outcomes, which carry the paired inference and the research-question claims, from *secondary* and *exploratory* outcomes, reported descriptively as controls and sensitivity analyses (Table 3). The primary outcomes are throughput and the closed-loop p99 tail on the five patterns against both engines at the fixed operating point; everything else bounds or contextualizes them.

Table 3: Outcomes and their analysis role. The primary outcomes carry the paired inference and the research-question claims; the secondary and exploratory outcomes are reported descriptively as controls and sensitivity analyses that bound or contextualize the primary results, and are not treated as additional confirmatory hypotheses.

Role	Outcome	Analysis
Primary	Throughput (req/s), 5 patterns × 2 engines, idiomatic layers, 50-connection operating point	Paired permutation, Wilcoxon, bootstrap median CI; spread & rank
Primary	Closed-loop p99 tail latency, same cells	Paired permutation, Wilcoxon, per-cell bootstrap CI on p99
Secondary	Same-SQL (raw-path) contrast	Bounds the residual (compound) same-SQL difference
Secondary	Sub-saturation open-loop tail (coordinated-omission corrected)	Lower-load latency companion to the high-load primary p99
Secondary	Layer×engine interaction (per pattern)	Blocked permutation (F on D)
Secondary	Combined app+db CPU efficiency, equal-CPU control	End-to-end compute view; operational confirmation of the CPU trade
Explor.	Aggregation fan-out; OFFSET vs. keyset; pool-size sweep; durability & isolation sensitivity; RSS	Descriptive; pitfall illustration and robustness checks

Because absolute throughput is hardware-specific, we analyze *relative* differences across layers within an engine. A pattern's sensitivity to the access layer is its *spread*: the idiomatic native driver's (`pg` or `mysql2`; the tuned baselines

are a separate reference, not the anchor) median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns even where an ORM exceeds the native driver). We report medians over the 25 repeated runs, the coefficient of variation (CV) as a stability signal following recommended practice [52] and using enough replicates to bound variance [53] (Section 4), and a bootstrap 95% CI for the median. The design is a randomized block: within each replicate every layer runs the identical request stream, so two layers' throughput samples are matched by replicate and compared with *paired* tests. To test whether adjacently ranked layers are distinguishable, we compare each adjacent pair on the deep/nested fetch on their per-replicate throughput ratios with a two-sided paired sign-flip permutation test (20,000 permutations, so the smallest attainable p is $1/(20{,}001)$), cross-checked with the Wilcoxon signed-rank test, reporting the geometric-mean paired ratio with a paired bootstrap 95% CI and the paired dominance (the fraction of replicates the faster layer wins). In every test the unit is the *run-level* statistic (one throughput or p99 value per repeated run, 25 per cell), never the individual request; all confidence intervals are *percentile* bootstraps resampling replicate indices (preserving the pairing), from a fixed seed (`mulberry32(0x57a75)`) so every resample is reproducible. Because p99 is reported at millisecond resolution and contains ties, the permutation test keeps zero log-ratios and the Wilcoxon test drops them with the standard tie correction (the paired p99 comparisons are Supplement Table S30). Because the seven adjacent pairs follow the *observed* ranking rather than a preregistered one, we treat their significances descriptively (post-hoc adjacent-rank comparisons), applying a Bonferroni correction as a conservatism check rather than claiming a confirmatory family. For the pg–Prisma pair we additionally report a paired two-one-sided-tests (TOST) equivalence result against a $\pm 5\%$ margin on the log-ratio (justified below). The layer$\times$engine interaction is tested per pattern with a blocked permutation test: for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference $D_{\ell,i}$ and, under the null of no interaction, permute the layer labels of $\{D_{\ell,i}\}$ *within* each replicate i (20,000 permutations); the statistic is the between-layer F of a

randomized-block ANOVA on D with replicate as the block. Rank agreement across engines (Spearman, Kendall) is reported descriptively over the seven evaluated layers — systems under study, not a sample from a population — so we attach no population confidence intervals to those correlations. Reporting nonparametric tests with standardized effect sizes follows recommended practice for randomized performance experiments in software engineering [54, 55]. The ±5% equivalence margin is an author-selected smallest effect of practical interest for *this* study, not a general engineering threshold: at modest deployment scale a sub-5% throughput difference would not change a capacity-planning decision (for example, how many application instances to provision for a target load). We set the margin from that engineering consequence rather than from measurement noise; it happens also to sit within the run-to-run and day-to-day variation typical of a virtualized host (this study's own run-to-run CV reaches 15.9% on the noisiest MySQL-insert cell, within 4.0% on the reads), but that coincidence is not the justification. At very large scale even 5% can correspond to many machines, so the margin should be reset per deployment. We chose it before the analysis but did not preregister it, and the equivalence conclusion is not sensitive to the exact value within a few percent.

3.7. Experimental setup

The measurements were taken on 2026-07-16 to 2026-07-18 on a single host: the primary matrix ran overnight and the order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, concurrency-sweep, utilization, pool-size, cluster, mixed-workload, wait-event, and post-restart runs followed over 2026-07-16 and 2026-07-17, with the longer-window tail re-measurement taken on 2026-07-18 (a virtualized Intel Xeon Gold 6140 instance, 16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), with the database engines (PostgreSQL 18.4, MySQL 9.7.1) running locally. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); a library is pinned to an earlier

release only where the latest stable breaks the contract or the byte-equivalence cross-check, and any such deviation is documented per layer. Only Sequelize invokes this exception: its version-7 line is a prerelease, so the current 6.x stable line is used. Prereleases and unreleased point versions are otherwise excluded. Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking, exactly one headline of which a superseded pin produced and a re-freeze retired (Section 4).

3.8. Replication package

The harness — the Express application, the adapter contract, all nine idiomatic adapters and the two tuned baselines, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts regenerating every table in Section 4 — is released so the full matrix can be re-run with one command against a containerized or local PostgreSQL and MySQL.

4. Results

The tables report throughput (requests per second, higher is better) and tail latency (p99, milliseconds, lower is better) per access layer and engine. The two consequential patterns are in the main text (the deep/nested fetch, Table 4, and the insert, Table 5); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15. All numbers come from the run of Section 3. Each native driver is reported both idiomatically (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking the throughput ceiling hand-tuned driver use reaches, not a level every idiomatic layer attains without code changes. We compare *relative* differences within an engine (Section 3); two estimands recur below: the *default-configuration (idiomatic) effect* (RQ1–RQ3) and the *same-SQL effect* (Supplement Table S14).

4.1. RQ1: performance difference relative to the native baseline

The native driver leads all ten engine-pattern combinations: on PostgreSQL `pg` (or its tuned counterpart) is fastest on all five patterns, and `mysql2` leads the

Table 4: Deep/nested fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3687 [3602–3737]	20 [18–21]	–	–
mysql2	native-driver	–	–	2382 [2357–2451]	28 [28–29]
pg-tuned	native-tuned	3746 [3684–3828]	18 [17–18]	–	–
mysql2-tuned	native-tuned	–	–	2415 [2387–2427]	25 [24–26]
knex	query-builder	2487 [2465–2505]	27 [26–28]	2007 [1989–2031]	30 [29–31]
drizzle	orm-lightweight	1865 [1837–1878]	35 [32–35]	1318 [1300–1336]	47 [46–48]
prisma	orm	1186 [1160–1203]	51 [51–53]	634 [618–644]	93 [89–95]
sequelize	orm	991 [977–998]	60 [59–61]	886 [868–898]	67 [66–68]
typeorm	orm	1204 [1188–1223]	50 [49–52]	1017 [987–1029]	57 [56–59]
objection	orm	1028 [1020–1037]	57 [56–58]	834 [828–855]	69 [67–71]
mikroorm	orm	516 [499–525]	116 [113–119]	480 [474–488]	120 [118–128]

Table 5: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine.

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6402 [6242–6547]	11 [10–15]	–	–
mysql2	native-driver	–	–	1753 [1717–1763]	113 [109–119]
pg-tuned	native-tuned	6413 [6073–6571]	11 [10–12]	–	–
mysql2-tuned	native-tuned	–	–	1750 [1482–1759]	120 [111–130]
knex	query-builder	4841 [4629–4908]	15 [14–16]	1670 [1470–1692]	127 [117–137]
drizzle	orm-lightweight	4085 [3928–4146]	15 [14–18]	1730 [1324–1745]	120 [109–138]
prisma	orm	2774 [2750–2815]	23 [22–24]	886 [828–903]	153 [137–160]
sequelize	orm	2732 [2719–2747]	23 [21–25]	1478 [1375–1684]	125 [118–132]
typeorm	orm	2648 [2630–2694]	23 [22–26]	1345 [1301–1359]	125 [119–129]
objection	orm	3907 [3746–3980]	17 [15–23]	1728 [1598–1746]	117 [108–132]
mikroorm	orm	1979 [1949–1997]	30 [29–31]	1257 [1240–1268]	117 [109–125]

two reads, the deep fetch, aggregation, and the insert on MySQL. The current-generation ORMs sit mid-to-low throughout, Prisma among the lowest: seventh of eight on both point reads, near the bottom of aggregation on both engines, and last on the MySQL insert (Tables 4, 5; Supplement Tables S12 and S15). This retires an earlier version's headline of Prisma tying the native driver at a large CPU premium (Section 5): no layer now exceeds about one application core (resource subsection below), leaving no CPU-for-throughput trade to buy parity with.

`pg-tuned` and `mysql2-tuned` track their idiomatic counterparts within a few percent on the single-round-trip patterns and edge them only on the deep fetch, where issuing more than one round trip lets reusable prepared statements and MySQL's binary protocol pay off (`pg-tuned` 3,746 vs. `pg` 3,687; `mysql2-tuned` 2,415 vs. `mysql2` 2,382); on PostgreSQL aggregation `pg-tuned` adds about 8% (7,538 vs. 6,978).

On the **point read**, the native-relative spread is 2.78× on PostgreSQL (`pg` 6,835 down to MikroORM 2,455) and 2.11× on MySQL (`mysql2` 4,368 down to MikroORM 2,067; Supplement Table S12); Prisma is next-to-last (3,255 / 2,074), ahead of only MikroORM.

On the **deep/nested fetch** (a post, its author, and all comments with their authors), the native-relative spread is the widest measured: 7.15× on PostgreSQL (`pg` 3,687, 95% CI [3,602–3,737], down to MikroORM's 516 [499–525]) and 4.96× on MySQL (`mysql2` 2,382 [2,357–2,451] down to MikroORM's 480 [474–488]). The native driver leads outright on both engines — no tie at the top — and the data-mapper ORMs, especially MikroORM, trail furthest (Table 4). The comparison is paired (Section 3); every adjacent step of the MySQL ranking separates under the paired permutation test (Table 6).

Aggregation is the least layer-sensitive pattern: the native-relative spread is only 1.83× on PostgreSQL and 1.72× on MySQL (Supplement Table S15), and the native driver leads both. The idiomatic difference is small when a layer forwards a single query and large only when it must materialize a nested object graph application-side, as on the deep fetch.

4.2. Same-SQL control

The *same-SQL control* (Section 3) bounds the residual difference once each layer runs the same statement rather than choosing its own query strategy: server-side capture confirms the statement text is byte-identical across layers on both engines while the wire protocols differ per layer, so this control is *same SQL, protocol disclosed*, not *same plan*. The contrast is composite — it changes query strategy, query count, prepared-statement behavior, the raw versus ORM API, and result marshalling together — so the residual difference is bounded but no single factor is isolated.

On the identical SQL, the deep-fetch native-relative spread collapses from its idiomatic 7.15× (PostgreSQL) and 4.96× (MySQL) to 1.68× and 2.01× (Supplement Table S14). Replacing each layer's idiomatic path with identical SQL bounds how much of the spread is attributable to query strategy and formulation *jointly*; because the intervention changes several mechanisms together, it does not separate strategy from execution machinery. On the raw path Drizzle rises from 1,865 to 3,237 req/s, Prisma from 1,186 to 2,155, and Knex from 2,487 to 2,926 on PostgreSQL, so layers that trail on the idiomatic fetch converge once they run the same statement. Prisma is the slowest layer on the raw path itself (2,155 on PostgreSQL, 1,206 on MySQL); we conjecture this reflects dispatch and marshalling overhead in its raw API rather than SQL execution, a hypothesis the composite contrast cannot confirm. MikroORM shows the largest idiomatic-to-same-SQL gap: its idiomatic path reaches only 0.16 (PostgreSQL) and 0.20 (MySQL) of its own same-SQL throughput, a 6.3× and 5.1× idiomatic-to-raw gap. A loading-strategy sensitivity check (Supplement Table S18) tests whether these deficits are merely a default-strategy artifact: switching the join-default data-mapper ORMs (Sequelize, MikroORM) to select-in, where the alternative is a byte-identical drop-in, makes them slower (0.73–0.80×), so their deficit persists under an alternative loading strategy; conversely, Objection's batched-select-in default is about 1.6× slower than a single join on MySQL (820 vs. 1,301 req/s), so part of *its* deficit there is a strategy choice rather than a fixed cost.

4.3. RQ2: engine dependence of the ranking

The relative ordering is largely, but not uniformly, stable across engines. The Spearman rank correlation between the PostgreSQL and MySQL orderings of the seven portable layers (descriptive; Section 3) is high on the reads — $\rho = 0.86$ on the point read, 0.89 on the range scan, and 0.86 on the deep fetch — but only *moderate* on aggregation ($\rho = 0.64$) and the insert ($\rho = 0.68$), which transfer less reliably though neither is uncorrelated. Prisma is the notable mover: mid-pack on PostgreSQL inserts (2,774 req/s, fourth of the seven portable layers) but last on MySQL inserts (886, behind even MikroORM's 1,257) — a rank flip invisible to a single-engine benchmark (Supplement Table S27). The cost's *magnitude* also depends on the engine: a blocked permutation test on the paired PostgreSQL-minus-MySQL log-throughput difference (Section 3) finds the layer×engine interaction significant on all five patterns (observed statistic above every one of the 20,000 permutations), so that p reaches its floor (1/20,001) and conveys little about magnitude; the magnitude (Supplement Table S28) is largest on the insert, where Prisma's 2,774 req/s on PostgreSQL becomes 886 on MySQL. Single-engine results are thus portable in order for reads but not in degree, most loosely for writes.

On the **keyset range scan** the native driver again leads, with a native-relative spread of 4.21× on PostgreSQL and 3.85× on MySQL (Supplement Table S13); the large nominal spread is driven disproportionately by MikroORM alone (905 / 856 req/s, far below every other layer), confirming that the earlier collapse observed under `OFFSET` pagination was a workload artifact rather than an engine or layer property (Section 5).

Inserts are the pattern where the engine dominates most visibly, under each engine's *default* durability configuration (Section 3). On MySQL the faster layers cluster tightly near an engine-imposed floor — `mysql2` 1,753, Drizzle 1,730, Objection 1,728, and Knex 1,670 req/s, within about 5% (Table 5) — with p99 around 113–153 ms; Prisma is the exception at 886 req/s (last), which widens the nominal MySQL spread to 1.98×. Direct `performance_schema` instrumentation attributes the floor to the commit path: the redo-log and binlog flush is

a shared engine cost, similar across the native driver, query builder, and slowest ORM despite their very different read performance, and removing it (relaxed durability) lifts `mysql2` from 1,576 to 3,238 req/s (Supplement Table S19). Relaxing durability roughly doubles MySQL insert throughput layer-wide (`mysql2` 2.35×, 1,753 to 4,114; Supplement Table S3), confirming the floor is engine-side rather than layer-specific. PostgreSQL, by contrast, spreads 3.23× (`pg` 6,402 down to MikroORM 1,979) and is only weakly durability-sensitive on the lean layers (`pg` 1.13× relaxed÷default; Supplement Table S3), so there the access layer, not the engine, remains the larger factor.

4.4. RQ3: which patterns matter most

Ranking the five patterns by native-relative spread on PostgreSQL gives deep fetch (7.15×) > range scan (4.21×) > insert (3.23×) > point read (2.78×) > aggregation (1.83×). On MySQL the head and tail of that order are unchanged — deep fetch (4.96×) > range scan (3.85×) at the top, aggregation (1.72×) last — while the insert and point read swap in the middle (point read 2.11×, insert 1.98×), the insert compressing because MySQL's engine floor, not the access layer, sets its ceiling (RQ2 above). The deep/nested fetch is thus the most consequential pattern on both engines and aggregation the least.

The primary deep-fetch workload uses posts with roughly ten comments on average. A fan-out sweep across 0, 1, 10, 50, 100, and 500 comments (both engines, medians of three repeated runs) shows the native-relative spread is *roughly constant* with graph size rather than growing with it, so on this schema the deep-fetch deficit behaves as a fixed multiplier. (An earlier three-run sweep had suggested a growing spread; that did not survive replication, so we report the fan-out as exploratory.)

4.5. Tail latency

We report the closed-loop p99 for every cell (p50, p90, p97.5 in the replication package) at the 50-connection operating point (Section 3), so the tail includes connection-acquisition queueing — the tail a deployment sees under load, not

the per-request latency of the open-loop experiment below. At this high-load point, tail latency tracks throughput: on the deep/nested fetch, PostgreSQL's p99 ladder runs from `pg` 20 to MikroORM 116 ms (each layer's p99 with a 95% CI, Table 4), monotonic below the native driver — a penalty throughput-only vendor benchmarks miss. A paired p99 analysis (Supplement Table S30) resolves every adjacent pair on both engines except the near-tied TypeORM–Prisma step on PostgreSQL (ratio 1.03, $p = 0.06$), mirroring the throughput ranking. On MySQL inserts, by contrast, the faster layers' tails cluster (113–127 ms) at the engine floor, with only Prisma's 153 ms standing out.

To check the tail ranking is not an artifact of the constant-concurrency model, we drove the deep/nested fetch on PostgreSQL under the coordinated-omission-corrected open-loop harness (Section 3; timed-out requests capped at 10 s and clipped into the distribution; full sweep in Supplement Table S6). Below saturation the corrected tails are flat and small (`pg` holds p99 at 2–3 ms up to 2,000 req/s; Knex 2 ms and Prisma 13 ms up to 1,000 req/s); beyond each layer's capacity the tail collapses rather than degrading gracefully, the data-mapper ORMs earliest. At 2,000 req/s Prisma and Sequelize exceed 10 s while `pg` still clears at 3 ms and Knex at 89 ms, and at 4,000 req/s `pg`'s corrected p99 is 1.48 s against Knex's 8.8 s and the ORMs above 10 s. Calling a *collapse* an offered rate at which a layer clears under half the load, the open-loop ordering matches the closed-loop one, and the MySQL sweep (Supplement Table S17) reproduces it, so the high-load-tail ordering is engine-robust. Offering each layer 50/70/85/95% of *its own* saturating throughput (coordinated-omission-corrected, five runs, both engines; Supplement Tables S21–S22), the tails converge: at 50% utilization every layer holds p99 near 2–5 ms on both engines regardless of its high-load rank, rising only as it nears its own capacity (for example Objection's p99 reaches 285 ms at 95% of its own capacity on PostgreSQL). At matched utilization the large high-load gap (`pg` 20 ms versus MikroORM 116 ms) therefore largely dissolves; it is a capacity-and-queueing effect, not a difference in tail latency at comparable utilization. The single-rate open-loop cells above are single runs, so near-knee values are indicative; the

utilization sweep and the flat-then-collapse ordering are what reproduce.

4.6. Measurement stability and significance

Across the 25 repeated runs the coefficient of variation of deep-fetch throughput — the pattern that carries the pairwise tests — is at most 4.0% on either engine (Supplement Tables S1 and S9), well below the between-layer spreads above; dispersion is larger only on the insert, where the MySQL cells are the noisiest, bimodal under group-commit batching (up to 15.9%). Every measured request in the primary matrix succeeded: HTTP errors, connection-acquisition timeouts, and non-2xx responses were zero across all 90 cells' measurement windows (nonzero timeouts arise only in the deliberate open-loop overload runs below). Those non-2xx counters are load-bearing: on the current versions MikroORM 7 removed `persistAndFlush`, so every MikroORM *write* threw an HTTP 500 — which returns faster than a real insert, so the broken cells briefly showed *inflated* throughput and spuriously led the MySQL insert until the non-2xx counts flagged them (Section 5). As an environmental-robustness check we restarted both database engines (cold buffer pools, fresh connections) and re-measured a representative deep-fetch subset (`pg`/`mysql2`, Prisma, MikroORM): the layer ranking reproduced the primary campaign on both engines, with absolute throughput up to 16% lower after the restart (larger on PostgreSQL), a cold-buffer and run-to-run drift the relative analysis absorbs (Supplement Table S20).

Comparing each adjacently ranked pair on the MySQL deep/nested fetch with the paired permutation test (Section 3; Table 6), all seven pairs separate: six with the observed statistic above every one of the 20,000 permutations ($p = 1/20{,}001 \approx 5 \times 10^{-5}$) and the faster layer ahead in every replicate (dominance 1.00), all with ratios at least 1.15; the narrowest step, Sequelize over Objection, still separates (ratio 1.05, dominance 0.88, $p = 0.0001$). The Wilcoxon test agrees throughout (replication package), and a Bonferroni correction leaves all seven far below any conventional threshold. Native `mysql2` leads outright with no tie at the top. Because these adjacent pairs were selected after observing

the ranking, we read their separation descriptively rather than as independent confirmation of the ordering, and it covers the deep fetch only (Table 6), not the full cross-pattern ranking.

4.7. Scaling with concurrency

A concurrency sweep from 1 to 256 connections (Supplement Figure S2) shows the three-band ordering (native driver, then Knex and Drizzle, then the data-mapper ORMs) emerges by 8 connections and holds at every operating point above about 32; at a single connection the layers are far closer together (about 320–1,020 req/s) and Prisma is mid-pack throughout, never approaching the native driver. The 50-connection operating point sits above the knee for every layer on the deep fetch, so the RQ1 ranking is not an artifact of a single point.

4.8. Resource footprint

On the current library generation the application-side CPU cost is uniform (Supplement Tables S5 and S10): on the deep/nested fetch, measured at the same ten-connection pool as every other layer, every layer draws about one core (100–110% of one logical core, the maximum being MikroORM's 110%), so no layer buys throughput with extra application cores. What differs is the database-side load: the native `pg` driver runs at roughly 100% application CPU but issues lean SQL that drives the database harder — about 355% database CPU on the PostgreSQL deep fetch — whereas the ORMs sit at the same $\approx$ 100% application CPU and drive lower database CPU (on MySQL, `mysql2` draws 85% against 25–70% for the ORMs; Supplement Table S5). The native driver therefore leads on throughput and end-to-end compute efficiency at once: counting combined application-plus-database CPU per completed request, `mysql2` completes 1,254 req per combined-CPU-second on the MySQL deep fetch against Prisma's 384 and MikroORM's 356 (Supplement Table S5). There is no CPU-for-throughput trade: the slower layers are also less compute-efficient.

An equal-CPU control confirms no layer gains throughput from additional application cores. Confining the server process with `taskset` to 1, 2, or 4 cores (database and load generator on disjoint cores; Supplement Table S4) changes deep-fetch throughput by at most a few percent per layer: `pg` holds 3,689, 3,688, and 3,687 req/s at 1, 2, and 4 cores, Prisma 1,079, 1,052, and 1,219, and MikroORM 446, 495, and 522, so the per-layer differences are not an artifact of unequal core budgets. Bound within one process, throughput scales out horizontally instead: under a shared-port `node` cluster (Supplement Table S24) Prisma rises from 1,117 to 2,275 to 4,449 req/s at 1, 2, and 4 workers on PostgreSQL, reaching native-like aggregate throughput at four workers (`pg` 3,299 to 4,055 to 3,961, saturating near two workers as it becomes database-bound). Prisma's low single-process throughput is thus recoverable by running more processes, at roughly four times the cores.

Memory differences are smaller (Supplement Table S10): the query builder and the lightest ORMs are cheapest (Knex 215, Objection 221, and TypeORM 225 MB peak RSS), while Drizzle, Prisma, and MikroORM use more (297–356 MB). Per-instance resource use governs how many replicas a given load requires when sizing horizontally scaled deployments [56].

5. Discussion

Access-layer rankings can be version-sensitive; the instrument is the contribution.. One case here illustrates the point. An earlier campaign found Prisma (then version 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores. Re-running the *identical* harness on the latest stable releases at the 15 July 2026 freeze did not reproduce it: no layer now draws more than about one core, and Prisma has fallen to mid-to-low throughput (Section 4). The non-reproduction is a fact; its cause is a hypothesis: the vanished premium was Prisma-specific and only Prisma changed architecture, so its Rust-free rewrite is the likeliest driver, but the re-freeze moved the whole toolchain at once, so we cannot isolate it. The

Table 6: Paired comparison of adjacently ranked access layers on the deep/nested fetch (PostgreSQL), over the 25 repeated runs. Because every layer runs the identical per-replicate request stream, layers are compared on their per-replicate throughput ratios: median throughput (req/s) with bootstrap 95% CI, the geometric-mean paired ratio A/B with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which A exceeds B), and the two-sided paired sign-flip permutation p (20000 permutations; a value of 5.0×10^{-5} is the resolution floor $1/(B+1) = 1/20{,}001$, i.e. the observed statistic exceeded every permutation; the Wilcoxon signed-rank test agrees, replication package).

Comparison (A > B)	med. A [95% CI]	med. B [95% CI]	ratio A/B [95% CI]	dom.	perm. p
pg > knex	3687 [3602–3737]	2487 [2465–2505]	1.48 [1.46–1.50]	1.00	5.0×10^{-5}
knex > drizzle	2487 [2465–2505]	1865 [1837–1875]	1.33 [1.31–1.35]	1.00	5.0×10^{-5}
drizzle > typeorm	1865 [1837–1878]	1204 [1188–1223]	1.54 [1.52–1.56]	1.00	5.0×10^{-5}
typeorm > prisma	1204 [1188–1223]	1186 [1160–1203]	1.02 [1.00–1.03]	0.64	0.0230
prisma > objection	1186 [1160–1203]	1028 [1020–1037]	1.15 [1.13–1.18]	1.00	5.0×10^{-5}
objection > sequelize	1028 [1020–1037]	991 [977–998]	1.04 [1.03–1.05]	0.88	5.0×10^{-5}
sequelize > mikroorm	991 [977–998]	516 [499–525]	1.93 [1.90–1.97]	1.00	5.0×10^{-5}

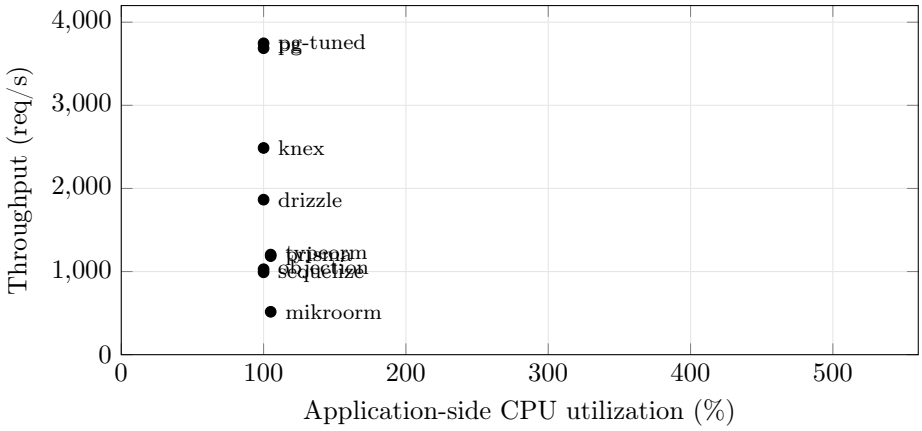

Figure 1: Throughput versus application-side CPU on the deep/nested fetch (PostgreSQL, ten-connection pool, medians of 25 replicates). Every layer sits at about 100–110% of one application core; the native driver leads on the throughput axis, and the slower ORMs are low on throughput at the same CPU. CPU is the server process only; database-side CPU is excluded for all layers alike.

headline is therefore version-sensitive [28], and the durable contributions are the reusable harness, the benchmark design, and the pitfalls checklist [57].

Implications for practice.. RQ1 and RQ3 agree that the idiomatic implementation-and-strategy difference is specific to the access *pattern*, concentrating where a layer must hydrate rows into objects and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [16] made concrete): the native-relative spread is widest on the deep/nested fetch and narrowest on aggregation, where every layer forwards the same correlated-subquery plan (Section 4). Round-trip count alone does not set throughput: on the PostgreSQL deep fetch the single-query MikroORM is the slowest layer of all while the two-query Drizzle outruns the four-query Prisma (Supplement Table S2; Table 4), so how a layer materializes a result graph matters more than how many statements it issues.

The same-SQL control.. The same-SQL control (Section 4) narrows the idiomatic deep-fetch spread substantially once every layer runs identical SQL; but because it changes query formulation, round-trip count, prepared-statement behavior, wire protocol, and result marshalling together, it bounds the combined contribution of query strategy and formulation without isolating a single factor such as eager-loading strategy.

Compute cost and horizontal scaling.. Under current versions the throughput differences are no longer bought with extra application cores. On the deep fetch every layer sits at roughly one application core — Prisma at about 105%, the native driver at about 100% (Figure 1) — and the native driver's lead comes instead from issuing lean SQL that pushes work onto the database tier (its PostgreSQL database-side CPU reaches about 355% against Prisma's 60%). The equal-CPU control confirms extra application cores do not raise a process's throughput (Section 4), so a layer's low per-process throughput is not a fixed ceiling: Prisma scales out nearly linearly under a shared-port `node` cluster to

native-like aggregate throughput at four workers (Section 4) — at four times the processes and cores, trading footprint and energy for throughput [29].

Queueing under load and the open-loop check.. The open-loop cross-run (Section 4) shows that a closed-loop benchmark alone would underestimate how abruptly, not just how much, a heavyweight ORM's tail degrades once offered load crosses its capacity.

Writes: a partly engine-bound cost under default durability.. RQ2 has a clear answer on inserts, under each engine's *default* durability configuration (Section 3): the native driver leads on both engines, and PostgreSQL's insert throughput stays strongly layer-dependent even with fsync on (3.23×; Table 5), whereas on MySQL most layers cluster within about 5% near an engine-level commit floor, consistent with the per-commit double flush (redo log, then binlog) InnoDB performs on every insert [44]; relaxing durability does not overturn this engine contrast (Section 4). The write ranking transfers only moderately across engines (Spearman $\rho = 0.68$, against 0.86–0.89 for the reads), so a single-engine write comparison invites the wrong generalization either way ("the access layer barely matters to writes" from MySQL, or "it matters a great deal" from PostgreSQL); an engine-only benchmark such as [12] cannot expose this interaction, since it omits the access layer entirely, whereas the dual-engine design does, consistent with earlier ORM work showing database technology modulates mapping-strategy cost [27]. The single-row insert is not the only write shape: a transactional multi-statement write, inserting a post and five comments in one transaction, spreads the representative layers about 3.7× (native `pg` 2,718 down to Prisma 733 req/s; Supplement Table S8), below the widest read spread, because the transaction's single commit amortizes the per-commit flush across six inserts. Prisma, mid-pack on the single-row PostgreSQL insert, falls to the bottom of the transactional subset, so the transactional ordering does not simply track the single-row one.

Methodological lessons: a checklist for access-layer benchmarking.. Reaching the figures above meant confronting four confounds that would otherwise have corrupted the comparison silently. One, the aggregation fan-out, is a *correctness* bug our automated byte-equality cross-check caught, along with a driver result-shape mismatch and — on the write path, which that read-oriented check does not cover — a fresh, current instance: MikroORM 7 removed `persistAndFlush`, so every MikroORM insert threw a 500, and because an error response returns faster than a real insert, the broken cells briefly appeared to *lead* the MySQL write ranking until the runner's non-2xx response counter flagged them and excluded the cell. Timing is meaningless until every response is a real 2xx, so the error counter is the write path's correctness gate as byte-equality is the read path's. The other three confounds return *correct* results, invisible to any equality check, and were found only by performance analysis. We regard the resulting checklist as reusable:

- **Aggregation fan-out.** A naive join between `posts` and `comments` evaluated before the `SUM` multiplies each post's view count by its number of matching comment rows, inflating the aggregate, a correctness bug invisible unless every layer's output is checked against a reference, not a performance one.
- **OFFSET vs. keyset pagination.** An early, OFFSET-based design for the range-scan endpoint collapsed on MySQL at realistic offsets, which would have been misread as an access-layer weakness rather than what it is: a pagination-strategy cost that InnoDB pays more dearly for than PostgreSQL. Rewriting every adapter around keyset pagination (`WHERE id < cursor ORDER BY id DESC LIMIT n`) removed the confound.
- **Write-workload contamination.** The insert endpoint grows `posts` on every call; because engines and layers commit at different rates, later cells in a run would scan a differently sized table depending on run order rather than on layer identity, unless benchmark-inserted rows are deleted before every cell.
- **Query-formulation confounds.** A first attempt at the aggregation endpoint pre-aggregated *all* of `comments` before filtering to one author, so every

request re-scanned the whole table regardless of which author was requested, a query-plan artifact masquerading as an access-layer effect [58], fixed by rewriting the query as a correlated subquery touching only the target author's rows.

None of these four is schema-specific; each is a generic failure mode of the genre, extending the database-benchmarking pitfalls cataloged by Fruth et al. [7] and Raasveldt et al. [32] to the access-layer-comparison sub-genre. Each is easy to introduce by accident and hard to detect (the correctness ones without an automated cross-check, the performance ones without profiling every cell), and at least one plausibly explains part of the gap between some vendor-reported and independently verified figures in this space.

Practical guidance.. The practical synthesis follows from RQ1–RQ3, on the default-configuration estimand a practitioner faces, and applies to the benchmarked implementations in this configuration, workload, and host rather than to access-layer categories in general. The native driver leads throughput on every pattern on both engines, and the idiomatic ORM penalty concentrates on relation-materializing traffic, whereas read-simple patterns and aggregation, where a layer merely forwards SQL, spread far less (Section 4). Where a service's hot path is dominated by deep or nested fetches, the access layer is consequential, and the tested thin paths (the native driver and Knex) were the safer default here *for throughput and response-time p99*; whether that advantage generalizes to other implementations of those tiers is a hypothesis for broader study, not a category law. Where the workload is read-simple or write-bound, especially on MySQL, whose engine floor for single-row inserts dominates layer differences under default durability, the throughput cost is small, so a full ORM's productivity and type safety, not measured here, may reasonably decide the choice. A layer's low per-process throughput is not on its own disqualifying: Prisma, among the slowest layers here on several patterns, recovers native-like aggregate throughput by scaling out at four times the processes (Section 4), trading single-process efficiency for replica count. Second-level caching can absorb part of an ORM's read cost where the workload tolerates staleness [25].

These are performance findings only: the access-layer decision also turns on developer productivity, type safety, correctness, maintainability, and security, none of which this study measures, and which may outweigh the throughput differences reported here. Even as performance guidance, the recommendations rest on one schema, one host, a single-host local-database deployment under default write durability, and the pinned versions; and the same-SQL comparisons bound the idiomatic cost rather than licensing raw SQL in practice. Section 6 sets out how far these results generalize.

6. Threats to Validity

We organize the threats along the four standard validity categories [59, 42].

Construct validity.. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip a deployed service incurs (Express routing, JSON serialization), though the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We hold them near-constant: every adapter satisfies a documented *adapter contract* and funnels its rows through shared canonical constructors (a fixed field set, key order, and value typing, under `TZ=UTC`), and the automated cross-check (`bench/verify.mjs`) asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run (Section 3). During development that check caught a defect: PostgreSQL's schema declared `created_at` as a timezone-naive timestamp although the column is `timestamptz`, so Drizzle's PostgreSQL responses carried instants shifted by the host's UTC offset — invisible to throughput and to any coarser equivalence check; it was fixed (`withTimezone: true`) before the campaign reported here. The fixed-payload baseline floor sits well above the fastest measured cell (Section 3), so the framework floor compresses, rather than manufactures, between-layer differences, which are therefore attributable to how each layer reaches an

identical response, not to what is returned. Writes are the one endpoint that mutates state; the reported finding uses each engine's *default*, vendor-standard durability configuration (Section 3), not one we relaxed ourselves, so the write spreads (Table 5) describe a standard deployment rather than a tuning choice; a symmetric relaxed-durability secondary confirms the cross-engine gap is not an artifact of either configuration (Section 5, Supplement Table S3).

Internal validity.. A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost; every adapter must use its layer's documented, idiomatic eager-loading strategy on the deep-fetch endpoint (Section 3). The byte-level correctness cross-check above caught two defects during development, both fixed before the measurements reported here (the checklist in Section 5). Explicit warm-up phases per cell (Section 3), set to exceed the slowest layer's cold-start stabilization, absorb JIT, pool-fill, and plan-cache effects, guarding against environmentally induced measurement bias [60]; two further controls rule out non-layer confounds — benchmark rows reset before every cell (physically rebuilt for writes), and one correlated-subquery plan for aggregation across layers (Section 3). Every layer and engine within a replicate is driven by the identical, PRNG-seeded id sequence, and a forward-versus-reversed cell-order check found no detectable history effect (Section 3), addressing the concern that processing order could confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [7, 50]; a native, coordinated-omission-corrected constant-arrival cross-run on the deep fetch (Section 3) confirms the tail ranking holds under that stricter model (Section 4; Supplement Table S6).

External validity.. All measurements come from a single schema, a single dataset size, one hardware configuration, and a connection pool fixed at ten, against specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1; Section 3). The fixed pool makes this an *equal configured connection limit* comparison, not a per-layer-tuned one: it isolates the access layer from pool tuning but does

not show each layer's throughput under its own best pool. A per-layer pool-size frontier on both engines (Supplement Tables S7 and S25) addresses this directly: the ordering is preserved from a pool of 1 to 50, and each layer's throughput saturates well below 50, so the fixed-pool choice does not drive the ranking. A mixed read/write workload (Supplement Table S26) likewise preserves the read ordering. Results may not transfer to larger working sets, other pool sizes, or other engine versions, and library performance ages quickly, so rather than freeze an arbitrary snapshot the study pins the latest stable release of each library compatible with the harness at the freeze date (2026-07-15; Supplement Table S11). Prisma is therefore measured at 7.8, the Rust-free pure-TypeScript client that replaced the earlier in-process Rust query engine [61], so the results characterize the current architecture rather than a superseded one; an earlier campaign on the Rust-engine generation had reported a multi-core application-CPU profile for Prisma that this re-run retires (Section 5). Two version choices are disclosed rather than resolved: Sequelize is measured on its stable 6.x line because the version 7 line is a prerelease at the freeze date, and Prisma's MySQL path runs through the MariaDB driver adapter (`@prisma/adapter-mariadb`), since Prisma ships no `mysql2` adapter — an asymmetry with the other MySQL layers. Pinning does not remove the currency risk, because libraries keep moving; what re-establishes a current ranking is the instrument, not the table, as this re-run shows. The host is a virtualized, 16-vCPU, single-NUMA-node cloud instance rather than dedicated bare metal, so absolute req/s figures reflect that substrate specifically; only the *relative* ordering within an engine is intended to travel. The working set is memory-resident, so these results are a hot-cache regime maximizing the visibility of layer differences; as a workload becomes I/O-bound (larger than RAM, cold caches), every layer increasingly waits on the same storage and the spreads should compress toward 1×. The deep-fetch object graph is small (a post with on average ten comments, at most 25 in the seed); the replicated fan-out sweep (Section 4) shows the spread is a roughly fixed multiplier from 0 to 500 child rows, so the headline figure is representative of the range rather than specific to one graph size. We mitigate these

limitations by pinning every version, publishing the seed and an automated environment capture, and releasing the harness for re-runs on other hardware or against newer releases. A further limitation is that the load generator and server share a host, common practice in this genre that removes network variance as a confound but can understate network-bound effects; process separation on one host is not hardware isolation, since the primary campaign did not pin application, database, and generator to disjoint cores, so scheduler and shared-cache contention between them is possible (the environment capture records governor, NUMA, and affinity). A resource-isolation check re-ran the representative deep-fetch cells with the three components pinned to disjoint core sets (`ISO_` run, replication package): the isolated and shared-host medians agree within about 2% (ratios 0.98–1.01 across pg, Prisma, and MikroORM), and the layer ordering is unchanged, so same-host contention is not driving the reported ranking. The equal-CPU control (Section 4, Supplement Table S4) shows no layer's deep-fetch throughput changes materially from one to four cores, so the ranking is not an artifact of unequal core access. A single-process caveat still extends to deployment topology: the server runs as a single Node.js process, whereas production deployments typically run one worker per core (`cluster`, pm2, per-pod replicas). A multi-worker check (Supplement Table S24) shows the low-per-process layers scale out near-linearly, so a layer's single-process throughput is not its aggregate ceiling, reaching native-like aggregate throughput at four times the processes (Section 4). A sustained-load check bounds temporal drift: holding three representative layers (native `pg`, Prisma, MikroORM) under ten minutes of continuous deep-fetch load, re-run under the final harness, moves throughput by at most $\pm 1.4\%$ between the first and last three minutes, with the band ordering preserved in every single minute, and a 90 s run reproduces the 12 s medians within 1% (replication package), so the short measured runs are not sampling a transient; longer-horizon drift over hours to days [4] remains untested. A two-machine cross-run over a dedicated link, adding real network latency, and a multi-worker variant are planned to bound this further.

Conclusion validity.. Our analysis (Section 3) reports medians with the coefficient of variation and bootstrap 95% confidence intervals as stability signals and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with the Wilcoxon signed-rank test; Table 6). Two bounds reinforce the ordering (Section 4): the widest spreads dwarf the run-to-run CV — deep-fetch throughput varies by at most about 4.0% across the 25 repeated runs on either engine (Supplement Tables S9 and S1) — and the layer ordering holds across the concurrency sweep at ≥ 32 connections (Supplement Figure S2). On the deep/nested fetch all seven adjacently ranked pairs separate (Table 6): six at the permutation-resolution floor ($p = 1/20{,}001$) with the faster layer ahead in every replicate, and the closest pair — Sequelize over Objection, a ratio of 1.05 — at $p = 10^{-4}$ with the faster layer ahead in 88% of replicates; all seven survive a Bonferroni correction across the post-hoc adjacent-rank comparisons (Section 4). Rather than assert a specific statistical power, which would require a prospective effect-size and variance model we did not prespecify, we rest the conclusions on effect sizes and interval estimates: the separating pairs combine large paired ratios with the permutation floor and non-overlapping bootstrap intervals, above the run-to-run dispersion. Booting a fresh process per replicate prevents persistent process state from carrying across replicate runs [62], but the replicates share one host and one campaign; we therefore treat replicate index and measurement order as blocks (Section 3) rather than claiming full independence, and more replicates or longer runs would tighten the intervals further.

7. Conclusion and Future Work

This paper replaces vendor-benchmark claims with a vendor-independent, reproducible comparison of relational database access layers in Express.js, serving an identical application through a representative taxonomy (native drivers, a query builder, and six ORMs) across PostgreSQL and MySQL, over five access

patterns, reporting throughput and response-time p99 together (Section 1).

The idiomatic implementation-and-strategy difference is concentrated, not uniform: sharpest for the deep/nested fetch that assembles a nested object graph application-side, and modest for point reads and aggregation (Section 4). The same-SQL control bounds this difference: running every layer on identical SQL narrows the spread substantially, but because that contrast changes several mechanisms together it does not attribute the difference to any single one (Section 4). On the current library versions no layer's throughput scales with additional application cores and the native driver leads all ten engine-pattern combinations; the read ordering is similar across the two engines while aggregation and writes transfer only moderately, a caution against generalizing single-engine results (Section 5). That access-layer rankings can be version-sensitive is, here, a measured result, not a hypothesis: an earlier version's headline did not survive re-running the same harness on newer versions. Development also surfaced a reusable checklist of benchmarking pitfalls specific to this genre, one of them, a fast-but-wrong write path that passed the read-level cross-check, added by this very re-run (Section 5).

These findings held under a concurrency sweep (1 to 256 connections) and under paired permutation and Wilcoxon tests confirming adjacent-layer separation (Section 4). Further extensions would deepen the evidence — a two-machine cross-run to bound observer interference beyond the open-loop check, additional engines and dataset sizes, and a dedicated study of the N+1 penalty and cold-start latency — and the released harness, exercised again by this revision's re-run against Prisma 7, makes them straightforward, addressing the repeatability gap documented in computer-systems research [36].

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table in this paper) is publicly available at `https://github.com/mmiotk/express-db-access-performance` and permanently archived at Zenodo as release v1.6.1 (`https://doi.org/10.5281/zenodo.21428215`).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with drafting and editing the manuscript and with implementing and analyzing the benchmark harness. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. All AI-assisted code was verified: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`, run by `npm test`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the byte-level correctness cross-check independently confirms that all layers return identical responses before any timing is trusted.

References

[1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.

[2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.

[3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.

[4] P. Kuffeł, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.

[5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.

[6] J. Dean, L. A. Barroso, The tail at scale, Commun. ACM 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.

[7] M. Fruth, S. Scherzinger, W. Mauerer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: Performance Evaluation and Benchmarking (TPCTC 2021), Vol. 13169 of Lecture Notes in Computer Science, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.

[8] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.

[9] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison of raw sql and prisma orm, Procedia Comput. Sci. 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.

[10] J. Attala, C. Khemapatpapan, Comparing the performance of prisma, typeorm, and sequelize on postgresql, J. Comput. Creat. Technol. 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16.
URL `https://so13.tci-thaijo.org/index.php/jcct/article/view/2330`

[11] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, J. Comput. Sci. Inst. 37 (2025) 426–430. doi:10.35784/jcsi.7951.

[12] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, Future Internet 16 (10) (2024) 382. doi:10.3390/fi16100382.

[13] Drizzle Team, Drizzle orm benchmarks, `https://orm.drizzle.team/benchmarks`, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).

[14] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, Procedia Comput. Sci. 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.

[15] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js,

https://github.com/mcollina/autocannon, version 7.15.0; accessed 10 July 2026. (2023).

[16] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: 2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA), IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.

[17] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, Inf. Softw. Technol. 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.

[18] P. J. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison-Wesley, Upper Saddle River, NJ, 2012.

[19] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.

[20] C. Yan, A. Cheung, J. Yang, S. Lu, Understanding database performance inefficiencies in real-world web applications, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 1299–1308. doi:10.1145/3132847.3132954.

[21] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, X. Wu, Database-access performance antipatterns in database-backed web applications, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 58–69. doi:10.1109/ICSME46990.2020.00016.

[22] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational map-

ping frameworks, IEEE Trans. Softw. Eng. 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.

[23] A. Cheung, A. Solar-Lezama, S. Madden, Optimizing database-backed applications with query synthesis, in: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2013, pp. 3–14. doi:10.1145/2491956.2462180.

[24] A. Cheung, S. Madden, A. Solar-Lezama, Sloth: Being lazy is a virtue (when issuing database queries), in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, ACM, 2014, pp. 931–942. doi:10.1145/2588555.2593672.

[25] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, P. Flora, CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), ACM, 2016, pp. 666–677. doi:10.1145/2950290.2950303.

[26] A. Turcotte, M. D. Shah, M. W. Aldrich, F. Tip, DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), ACM, 2022, pp. 774–785. doi:10.1145/3510003.3510097.

[27] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.

[28] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.

[29] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.

[30] Y. Li, S. Manoharan, A performance comparison of SQL and NoSQL databases, in: 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), IEEE, 2013, pp. 15–19. doi:10.1109/PACRIM.2013.6625441.

[31] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.

[32] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: Proceedings of the Workshop on Testing Database Systems (DBTest), ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.

[33] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.

[34] Z. M. Jiang, A. E. Hassan, A survey on load testing of large-scale software systems, IEEE Trans. Softw. Eng. 41 (11) (2015) 1091–1118. doi:10.1109/TSE.2015.2445340.

[35] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.

[36] C. Collberg, T. A. Proebsting, Repeatability in computer systems research, Commun. ACM 59 (3) (2016) 62–69. doi:10.1145/2812803.

[37] K. Huppler, The art of building a good benchmark, in: Performance Evaluation and Benchmarking (TPCTC 2009), Vol. 5895 of Lecture Notes in Computer Science, Springer, 2009, pp. 18–30. doi:10.1007/978-3-642-10424-4_3.

[38] S. E. Sim, S. Easterbrook, R. C. Holt, Using benchmarking to advance research: A challenge to software engineering, in: Proceedings of the 25th International Conference on Software Engineering (ICSE), IEEE, 2003, pp. 74–83. doi:10.1109/ICSE.2003.1201189.

[39] Prisma, Query performance benchmarks, `https://benchmarks.prisma.io/`, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).

[40] Gel Data (EdgeDB), Imdbench: Orm benchmarks, `https://github.com/geldata/imdbench`, accessed 10 July 2026. (2024).

[41] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

[42] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, 2nd Edition, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

[43] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Trans. Softw. Eng. 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.

[44] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: Proceedings of the 2008

ACM SIGMOD International Conference on Management of Data, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.

[45] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: 2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA), IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.

[46] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, Proc. VLDB Endow. 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.

[47] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html

[48] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, L. Tratt, Virtual machine warmup blows hot and cold, Proceedings of the ACM on Programming Languages 1 (OOPSLA) (2017) 1–27. doi:10.1145/3133876.

[49] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, Proc. VLDB Endow. 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

[50] G. Tene, How NOT to measure latency, https://www.infoq.com/presentations/latency-response-time/, conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026). (2015).

[51] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, https://arxiv.org/abs/2605.00428 (2026).

[52] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.

[53] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, Empir. Softw. Eng. 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.

[54] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, Softw. Test. Verif. Reliab. 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.

[55] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychol. Bull. 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[56] L. M. Vaquero, L. Rodero-Merino, R. Buyya, Dynamically scaling applications in the cloud, ACM SIGCOMM Comput. Commun. Rev. 41 (1) (2011) 45–52. doi:10.1145/1925861.1925869.

[57] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, A. Iosup, Methodological principles for reproducible performance evaluation in cloud computing, IEEE Trans. Softw. Eng. 47 (8) (2021) 1528–1543. doi:10.1109/TSE.2019.2927908.

[58] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proc. VLDB Endow. 9 (3) (2015) 204–215. doi:10.14778/2850583.2850594.

[59] T. D. Cook, D. T. Campbell, Quasi-Experimentation: Design and Analysis Issues for Field Settings, Houghton Mifflin, Boston, MA, 1979.

[60] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the

14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.

[61] Prisma, Prisma 7 performance and benchmarks, `https://www.prisma.io/blog/prisma-7-performance-benchmarks`, announces the query-engine rewrite from the Rust engine to a TypeScript client (accessed 13 July 2026). (2025).

[62] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 International Symposium on Memory Management (ISMM), ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.